

力扣 Hot100 题目思路速记（分类版·紧凑分页）

只保留面试中最常用、最容易被接受的主流思路

本文按常见面试题型重新分类整理，每题只保留一条最主流、最容易在面试里讲清楚思路。这一版不再按“一类一页”强制分页，而是采用更紧凑的分页方式：能和上一类放在同一页时就尽量合并，放不下时再整体换页，以减少空白页带来的浪费。

数组、字符串与矩阵

5. 最长回文子串：（1）以每个位置为中心向两边扩展，分别处理奇数和偶数长度回文（一个额外函数即可），取最长即可。（2） $dp[i][j] = (s[i] == s[j]) \text{ and } dp[i + 1][j - 1]$

48. 旋转图像：（1）四元组交换顺序（2）先沿主对角线翻转，再把每一行左右翻转，就得到顺时针旋转九十度的结果。

238. 除自身以外数组的乘积：先算每个位置左边乘积，再从右往左乘上右边乘积

240. 搜索二维矩阵 II：从右上角开始，当前值大了就往左，小了就往下。

283. 移动零：双指针，慢指针指向下一个该放非零元素的位置，快指针负责扫描。

448. 找到所有数组中消失的数字：循环访问num，将位置num-1的值加上n，最终 $\leq n$ 的为答案

581. 最短无序连续子数组：从左往右找右边界，从右往左找左边界；一旦当前值破坏最值单调性，就更新边界。

647. 回文子串：以每个位置为中心向两边扩展，分别处理奇数和偶数长度回文。

哈希与前缀和

1. 两数之和：哈希表记录已经遍历过的数，遍历到当前数时先查目标值减当前数是否出现，再把当前数下标存入表中。

49. 字母异位词分组：把排序后的字符串作为哈希表键（哈希表只能存储元组、字符串等不可变），相同键的原字符串放到同一组。

128. 最长连续序列：把所有数放入哈希集合，只从连续段起点开始向后枚举（如果num-1在里面就跳过），这样每个数只会被访问一次。

437. 路径总和 III：前缀和+DFS+回溯，统计从根到当前路径上有多少前缀和满足差值等于目标值。

560. 和为 K 的子数组：前缀和加哈希表，统计之前出现过多少个前缀和等于当前前缀和减 K。

链表

2. 两数相加：同时遍历两个链表并维护进位，逐位相加后接到新链表末尾。

19. 删除链表的倒数第 N 个结点：快指针先走 N 步，再和慢指针一起走，快指针到尾时慢指针正好在待删节点前一个位置。

21. 合并两个有序链表：双指针比较两个链表当前节点，把较小节点依次接到结果链表后面。

23. 合并 K 个升序链表：(1) 用小根堆heapq存储所有节点。(2) 分治合并

141. 环形链表：快慢指针，快指针每次走两步，慢指针每次走一步，能相遇就说明有环。

142. 环形链表 II：快慢指针相遇后，一个指针回到头节点，和慢指针同步前进，再次相遇的位置就是入环点。

148. 排序链表：链表适合归并排序，先快慢指针找中点拆成两半，再分别排序后合并。

160. 相交链表：两个指针分别走完自己的链表后切到对方链表头，第二次遍历时会在交点相遇。

206. 反转链表：用前驱和当前节点两个指针，逐个修改 next 指针方向。

234. 回文链表：找中点、反转后半段链表、前后两段逐个比较。

双指针与滑动窗口

3. 无重复字符的最长子串：滑动窗口加哈希表记录字符最近位置，左边界始终跳到重复字符上次出现位置的右边。

11. 盛最多水的容器：双指针从两端向中间收缩，每次移动较短的那一边，因为面积上限由短板决定。

15. 三数之和：先排序，固定第一个数后用左右指针找另外两个数，过程中跳过重复元素。

42. 接雨水：双指针维护左右两侧目前最高柱子，哪边低就先结算哪边能接多少水。

76. 最小覆盖子串：滑动窗口统计窗口内字符出现次数，满足覆盖后尽量收缩左边界。

438. 找到字符串中所有字母异位词：定长滑动窗口维护字符计数，窗口长度达到模式串长度后比较是否匹配。

栈、单调栈与队列

20. 有效的括号：用栈维护未匹配的左括号，遇到右括号就检查栈顶是否能配对。

32. 最长有效括号：栈里存下标，先放一个哨兵下标，遇到右括号时弹栈并用当前下标减栈顶更新答案。

84. 柱状图中最大的矩形：单调栈维护递增高度，遇到更矮柱子时不断出栈并计算以出栈高度为高的最大矩形。

85. 最大矩形：把每一行都看成柱状图的底边，累计每列连续一的高度，再调用上一题的单调栈做法。

155. 最小栈：用辅助栈同步记录当前栈内最小值，这样取最小值就是栈顶。

239. 滑动窗口最大值：单调队列维护窗口内可能成为最大值的下标，队首始终是当前窗口最大值。

394. 字符串解码：用栈处理数字和括号，遇到右括号时弹出本层字符串并按次数展开。

二分查找

- 4. 寻找两个正序数组的中位数：用二分在较短数组上做划分，让左半部分总长度固定，满足左边最大值不大于右边最小值后直接取中位数。
- 33. 搜索旋转排序数组：二分判断哪一半有序，再看目标值是否落在有序区间内，决定往哪边继续找。
- 34. 在排序数组中查找元素的第一个和最后一个位置：分别做两次二分，第一次找左边界，第二次找右边界。
- 35. 搜索插入位置：标准二分，最终左边界就是目标值应插入的位置。

回溯

- 17. 电话号码的字母组合：回溯按位选择，每一位从对应字母表里枚举一个字符加入路径。
- 22. 括号生成：回溯维护左右括号已使用数量，左括号没用完就放左括号，右括号数量小于左括号时才能放右括号。
- 39. 组合总和：回溯枚举每个数选几次，因为元素可重复使用，所以递归时下一层仍可从当前下标开始。
- 46. 全排列：回溯加使用标记，每一层从还没用过的数里选一个放入当前位置。
- 78. 子集：回溯，每个数都只有选或不选两种决策，顺着下标递归即可。
- 79. 单词搜索：从每个起点做 DFS，走过的格子临时标记为已访问，四个方向继续搜索。

排序与堆

56. 合并区间：先按左端点排序，再顺序扫描，当前区间能和上一个结果区间重叠就合并，否则新开一个区间。

75. 颜色分类：三指针做荷兰国旗问题，左边放零，右边放二，中间自然是一。

215. 数组中的第 K 个最大元素：用大小为 K 的小根堆维护当前最大的 K 个数，堆顶就是第 K 大。

347. 前 K 个高频元素：先统计频次，再用大小为 K 的小根堆保留频次最高的 K 个元素。

406. 根据身高重建队列：先按身高从高到低排序，身高相同按前面人数从小到大排，再按人数下标插入。

贪心

55. 跳跃游戏：贪心维护当前能到达的最远位置，只要遍历过程中下标没有超过最远位置就继续更新。

121. 买卖股票的最佳时机：遍历时维护历史最低价格，并用当前价格减最低价更新最大利润。

169. 多数元素：摩尔投票，维护候选人和票数，遇到相同就加一，不同就减一。

621. 任务调度器：统计最高频次任务，答案是空桶公式和任务总数两者中的较大值。

动态规划

- 10. 正则表达式匹配：**动态规划，dp 表示两个前缀是否匹配，分类讨论普通字符、点号和星号的转移。
- 53. 最大子数组和：**动态规划或贪心，当前位置的最优和等于当前值与前一个位置最优和加当前值中的较大者。
- 62. 不同路径：**动态规划，走到某格的方法数等于上方和左方的方法数之和。
- 64. 最小路径和：**动态规划，走到某格的最小和等于上方或左方较小路径和加当前格值。
- 70. 爬楼梯：**本质是斐波那契，走到当前台阶的方法数等于前一阶和前两阶的方法数之和。
- 72. 编辑距离：**二维动态规划，分别考虑插入、删除、替换三种操作取最小值。
- 96. 不同的二叉搜索树：**动态规划，以每个节点作为根时，左子树方案数乘右子树方案数，再把所有根的位置累加。
- 139. 单词拆分：**动态规划，dp 表示前缀能否被字典拆开，枚举最后一个单词的起点做转移。
- 152. 乘积最大子数组：**同时维护以当前位置结尾的最大乘积和最小乘积，因为负数会让最大最小互换。
- 198. 打家劫舍：**动态规划，当前房屋只有偷或不偷两种选择，取前两种状态的最优值。
- 221. 最大正方形：**动态规划，当前位置为右下角的最大正方形边长等于左、上、左上三者最小值再加一。
- 279. 完全平方数：**动态规划，枚举最后一步减去哪个平方数，取最少步数。
- 300. 最长递增子序列：**维护一个 tails 数组，tails 长度就是答案，每个新数用二分替换第一个大于等于它的位置。
- 309. 最佳买卖股票时机含冷冻期：**动态规划维护持有、刚卖出、冷冻结束三种状态。
- 312. 戳气球：**区间动态规划，枚举区间内最后一个被戳破的气球，把大区间拆成左右两个小区间。
- 322. 零钱兑换：**完全背包，dp 表示凑出当前金额最少需要多少枚硬币。
- 337. 打家劫舍 III：**树形动态规划，返回偷当前节点和不偷当前节点两种状态。
- 416. 分割等和子集：**转成零一背包，看是否能选出若干数恰好装满总和的一半。
- 494. 目标和：**把加减号分配问题转成子集和问题，再用零一背包统计方案数。

二叉树

94. 二叉树的中序遍历：递归按左根右遍历；面试时也可以用显式栈模拟。
98. 验证二叉搜索树：中序遍历二叉搜索树一定严格递增，只要检查当前值是否大于前一个值即可。
100. 相同的树：同时递归比较两棵树当前节点的值以及左右子树是否都相同。
101. 对称二叉树：递归判断左子树的左边和右子树的右边是否相同，左子树的右边和右子树的左边是否相同。
102. 二叉树的层序遍历：队列按层 BFS，每次取出当前层所有节点并把下一层加入队列。
104. 二叉树的最大深度：递归返回左右子树最大深度加一，或用层序遍历统计层数。
105. 从前序与中序遍历序列构造二叉树：前序第一个节点是根，在中序里找到根的位置后递归构造左右子树。
108. 将有序数组转换为二叉搜索树：每次取数组中点作为根节点，递归构造左右两半区间，对应得到平衡二叉搜索树。
114. 二叉树展开为链表：后序处理，先把左右子树都展开，再把左子树接到右边并把原右子树接到新链表末尾。
124. 二叉树中的最大路径和：递归返回从当前节点往下走的一条最大贡献值，同时用左贡献加右贡献加根值更新全局答案。
226. 翻转二叉树：递归交换每个节点的左右子树。
236. 二叉树的最近公共祖先：递归向左右子树查找，若两边都找到，当前节点就是最近公共祖先。
297. 二叉树的序列化与反序列化：层序遍历序列化，空节点也写进去；反序列化时按队列把左右孩子重新接回去。
538. 把二叉搜索树转换为累加树：按右根左逆中序遍历，维护一个累加和，当前节点值改成累加后的结果。
543. 二叉树的直径：递归返回左右子树最大深度，同时用左深度加右深度更新直径。
572. 另一棵树的子树：枚举原树每个节点作为起点，递归判断两棵子树是否完全相同。
617. 合并二叉树：同时递归两棵树，同位置节点相加，空节点直接接另一棵树。

图、网格与搜索

200. 岛屿数量：遍历网格，遇到陆地就做 DFS 或 BFS 把整块岛屿淹掉，计数加一。
207. 课程表：拓扑排序，统计入度，把入度为零的课程入队，最后看能否处理完所有课程。
695. 岛屿的最大面积：DFS 或 BFS 统计每块岛屿面积，过程中把访问过的陆地标记掉并更新最大值。

设计题与数据结构

146. LRU 缓存：哈希表加双向链表，哈希表负责 $O(1)$ 找节点，双向链表维护最近使用顺序。
208. 实现 Trie（前缀树）：树节点保存若干子节点和单词结束标记，插入和查询都按字符逐层往下走。

位运算与技巧

136. 只出现一次的数字：所有数按位异或，成对出现的数会抵消，最后剩下的就是答案。
287. 寻找重复数：把数组看成链表，用快慢指针找环入口，重复数就是入环点。
338. 比特位计数：利用最低有效位，当前数的一的个数等于去掉最低位一后的结果再加一。
461. 汉明距离：先异或，再统计结果里有多少个一。